

```
*Main> :t sumInt 2
sumInt 2 :: Int -> Int
```

The value of `sumInt 2 3` is an `Int` as defined in the type signature. We can also partially apply the function `sumInt` with one argument and its return type will be `Int -> Int`. In other words, `sumInt 2` takes an integer and will return an integer with 2 added to it.

Every function in Haskell takes only one argument. So, we can think of the `sumInt` function as one that takes an argument and returns a function that takes another argument and computes their sum. This return function can be defined as a `sumTwoInt` function that adds a 2 to an `Int` using the `sumInt` function, as shown below:

```
sumTwoInt :: Int -> Int
sumTwoInt x = sumInt 2 x
```

The '=' sign in Haskell signifies a definition and not a variable assignment as seen in imperative programming languages. We can thus omit the 'x' on either side and the code becomes even more concise:

```
sumTwoInt :: Int -> Int
sumTwoInt = sumInt 2
```

By loading `Sum.hs` again in the GHCi prompt, we get the following:

```
*Main> :l Sum.hs
[1 of 1] Compiling Main             ( Sum.hs, interpreted )
Ok, modules loaded: Main.
```

```
*Main> :t sumTwoInt
sumTwoInt :: Int -> Int
```

```
*Main> sumTwoInt 3
5
```

Let us look at some examples of functions that operate on lists. Consider list 'a', which is defined as `[1, 2, 3, 4, 5]` (a list of integers) in the `Sum.hs` file (re-load the file in GHCi before trying the list functions).

```
a :: [Int]
a = [1, 2, 3, 4, 5]
```

The `head` function returns the first element of a list:

```
*Main> head a
1
```

```
*Main> :t head
head :: [a] -> a
```

The `tail` function returns everything except the first element

from a list:

```
*Main> tail a
[2,3,4,5]
```

```
*Main> :t tail
tail :: [a] -> [a]
```

The `last` function returns the last element of a list:

```
*Main> last a
5
```

```
*Main> :t last
last :: [a] -> a
```

The `init` function returns everything except the last element of a list:

```
*Main> init a
[1,2,3,4]
```

```
*Main> :t init
init :: [a] -> [a]
```

The `length` function returns the length of a list:

```
*Main> length a
5
```

```
*Main> :t length
length :: [a] -> Int
```

The `take` function picks the first 'n' elements from a list:

```
*Main> take 3 a
[1,2,3]
```

```
*Main> :t take
take :: Int -> [a] -> [a]
```

The `drop` function drops 'n' elements from the beginning of a list, and returns the rest:

```
*Main> drop 3 a
[4,5]
```

```
*Main> :t drop
drop :: Int -> [a] -> [a]
```

The `zip` function takes two lists and creates a new list of tuples with the respective pairs from each list. For example:

```
*Main> let b = ["one", "two", "three", "four", "five"]
```